



KLE Technological University

Creating Value,
Leveraging Knowledge

Dr. M. S. Sheshgiri Campus, Belagavi

Department of Electronics & Communication Engineering

Micro Architectural Design of Digital Systems

25EECE302

“Design of Prime Number Detection Algorithm in Assembly on 5-Stage Pipelined Processor”

Submitted by:

Name	SRN
Sagar Shirodkar	02FE23BEC062
Parth Jadhav	02FE23BEC083
Roopashree Pote	02FE23BEC122
Keerti Mahantshetti	02FE23BEC123

Mentored by:

Prof. Ashwini Desai

Prof. Sushant Jadhav

Dept. of Electronics & Communication Engineering,
KLE Technological University,
Dr. M. S. Sheshgiri Campus, Belagavi

Academic Year 2025–26

CERTIFICATE

This is to certify that the project work entitled ‘**Design of Prime Number Detection Algorithm in Assembly on 5-Stage Pipelined Processor**’ is carried out by **Parth Jadhav** (SRN: 02FE23BEC083), **Sagar Shirodkar** (SRN: 02FE23BEC062), **Roopashree Pote** (SRN: 02FE23BEC122), and **Keerti Mahantshetti** (SRN: 02FE23BEC123), the bonafide students of **VI Semester** of **KLE Technological University, Dr. M. S. Sheshgiri Campus, Belagavi**, in partial fulfillment for the award of the degree of **Bachelor of Engineering** in the Department of **Electronics and Communication Engineering** during the academic year **2025–2026**.

It is certified that all the corrections and suggestions indicated for internal assessment have been incorporated in the project report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of the project work prescribed for the said degree.

Guide Signature	Guide Signature	HOD Signature
Prof. Ashwini Desai	Prof. Sushant Jadhav	Dr. D. A. Torse
 Examiner – I		
 Examiner – II		

Contents

Project Title	3
Objective	3
Apparatus / Tools Required	3
Theory	3
Block Diagram	4
Procedure	20
Expected Output	20
Waveform Observation Points	21
Result and Conclusion	23
Reference	23

Project Title

Design of Prime Number Detection Algorithm in assembly on 5-Stage Pipelined Processor

Objective

The objective of this project is to design and implement a Prime Number Detection System using a 5-Stage Pipelined Processor in Verilog HDL. The processor is designed with the standard pipeline stages: Instruction Fetch, Instruction Decode, Execution, Memory, and Write Back. The main aim is to understand the working of pipelined processor architecture and apply it to perform prime number detection using arithmetic logic.

This project also focuses on verifying each pipeline stage through simulation using Cadence tools. The program counter, instruction memory, pipeline registers, register file, and ALU are implemented and tested using waveform outputs. The execution stage performs the modulus operation required to check whether a given number is prime or not.

The project helps in understanding RTL design, pipelined datapath implementation, instruction flow, and simulation-based verification of digital processor architecture.

Apparatus / Tools Required

- **Computer System:** Used for coding and simulation.
- **Linux Operating System:** Used to run Cadence tools.
- **Cadence NCLaunch / Xcelium:** Used for Verilog compilation and simulation.
- **SimVision:** Used to view and verify waveforms.
- **Verilog HDL:** Used for designing the processor modules.
- **Text Editor:** Used to write and edit Verilog code.
- **FPGA Board:** Can be used for future hardware implementation of the processor.

Theory

A pipelined processor is a processor architecture in which instruction execution is divided into multiple stages. The 5-stage pipeline consists of Instruction Fetch (IF), Instruction Decode (ID), Execution (EX), Memory Access (MEM), and Write Back (WB). Each stage performs a specific operation, allowing multiple instructions to be processed simultaneously and improving processor performance.

In this project, the 5-stage pipelined processor is used for prime number detection. The execution stage performs the modulus operation to check divisibility of the given number. If the remainder becomes zero for any divisor other than 1 and the number itself, the number is declared non-prime; otherwise, it is considered prime.

Flow Diagram

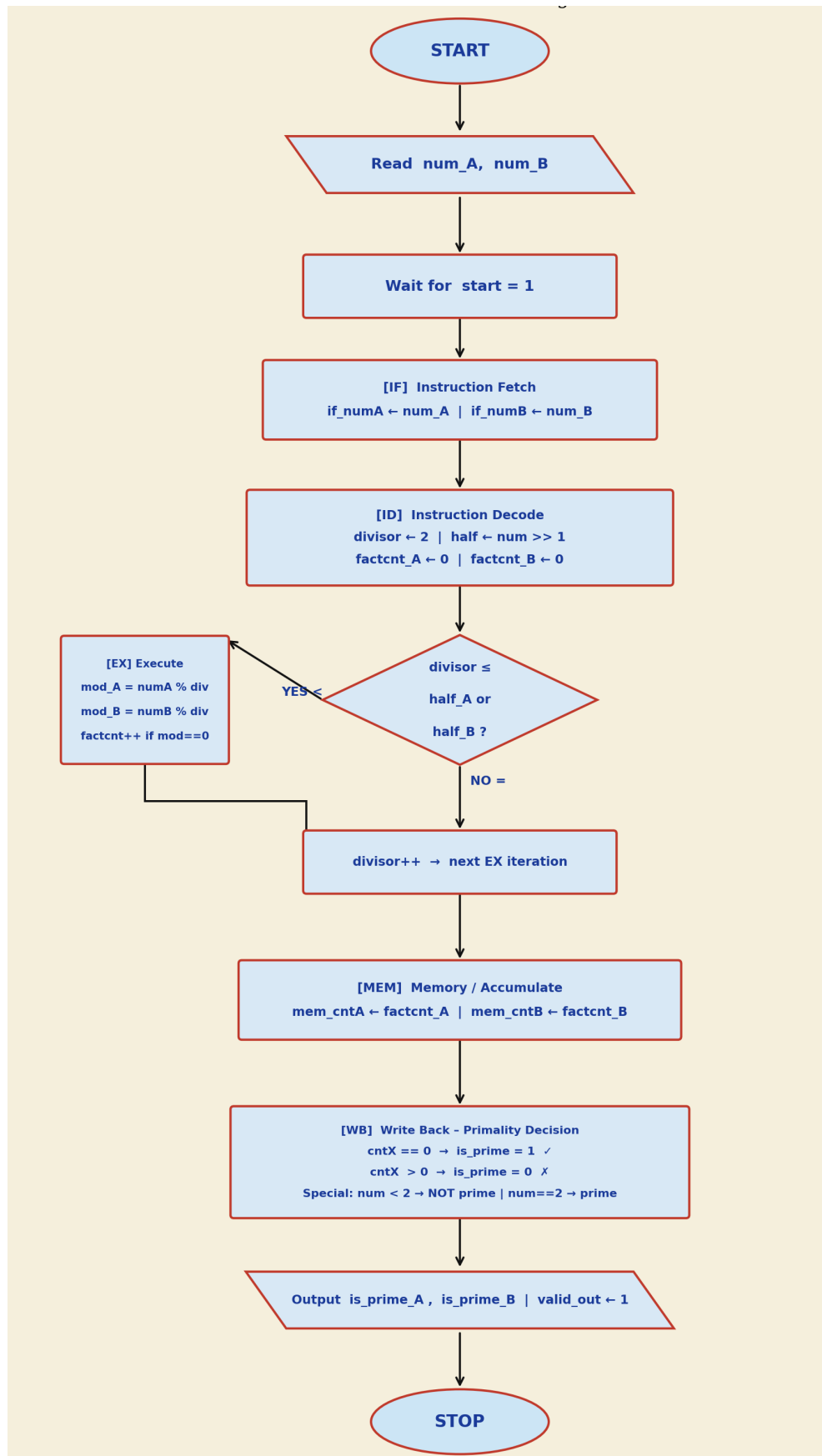


Figure 1: Prime detection flow diagram

Algorithm

The algorithm used for the 5-stage pipelined prime number detection processor is based on checking the divisibility of the given numbers using trial division.

1. Start the process.
2. Read the input numbers N_A and N_B .
3. Wait for the `start` signal.
4. In the **Instruction Fetch (IF)** stage, load the input numbers into pipeline registers.
5. In the **Instruction Decode (ID)** stage:
 - Initialize divisor as 2.
 - Calculate upper bound as $N/2$.
 - Prepare working registers.
6. In the **Execute (EX)** stage:
 - Perform division check remainder using mod operation.
 - Increment factor count if remainder is zero.
 - Increment divisor value.
 - Repeat until divisor exceeds $N/2$.
7. In the **Memory/Accumulate (MEM)** stage:
 - Store factor counts.
 - Prepare primality result.
8. In the **Write Back (WB)** stage:
 - Declare prime if no factors are found otherwise non-prime.
 - Generate `is_prime_A` and `is_prime_B` outputs.
9. Assert the `valid_out` signal.
10. Return to idle state.
11. Stop the process.

Codes

1. OPCode

```
// isa_pkg.v | Custom ISA opcode definitions
// Include this in every module that decodes instructions

// Instruction format: 16-bit
// [15:12] = OPCODE (4 bits)
// [11:9] = RD (destination register, 3 bits → 8 regs)
// [8:6] = RS1 (source register 1)
// [5:3] = RS2 (source register 2)
// [2:0] = IMM (3-bit immediate, sign-extended when needed)

`define OP_LOAD 4'b0001 // LOAD RD, IMM | load 8-bit immediate into RD
`define OP_MOV 4'b0010 // MOV RD, RS1 | copy RS1 into RD
`define OP_ADD 4'b0011 // ADD RD, RS1, RS2
`define OP_SUB 4'b0100 // SUB RD, RS1, RS2
`define OP_MOD 4'b0101 // MOD RD, RS1, RS2 | RD = RS1 % RS2
`define OP_SHR 4'b0110 // SHR RD, RS1 | RD = RS1 >> 1
`define OP_CMP 4'b0111 // CMP RS1, RS2 | sets zero_flag if
RS1==RS2
`define OP_BEQ 4'b1000 // BEQ IMM_ADDR | branch if zero_flag==1
`define OP_BGT 4'b1001 // BGT IMM_ADDR | branch if RS1 > RS2
`define OP_INC 4'b1010 // INC RD | RD = RD + 1
`define OP_OUT 4'b1011 // OUT RS1 | drive output port
`define OP_HALT 4'b1111 // HALT | stop, assert valid_out
`define OP_NOP 4'b0000 // NOP

// Register aliases (use these in assembly comments for readability)
// R0 = 0 (always zero, read-only by convention)
// R1 = num_A input
// R2 = num_B input
// R3 = divisor
// R4 = half (num/2)
// R5 = factcnt_A
// R6 = factcnt_B
// R7 = temp / scratch
```

2. Assembly Program

```
// =====
// This IS your "assembly program" stored as machine code.
// ; STAGE IF/ID: Initialise
// ADDR 0: LOAD R3, 2 ; divisor = 2
// ADDR 1: SHR R4, R1 ; half_A = num_A >> 1
// ADDR 2: SHR R7, R2 ; half_B = num_B >> 1 (R7 = half_B)
// ADDR 3: LOAD R5, 0 ; factcnt_A = 0
```

```

// ADDR 4:  LOAD  R6, 0          ; factcnt_B = 0
//
// ; STAGE EX: Trial Division Loop
// ADDR 5:  CMP   R3, R4          ; compare divisor vs half_A
// ADDR 6:  BGT   16              ; if divisor > half_A → jump to ADDR 16
// (check B done)
// ADDR 7:  MOD   R0, R1, R3      ; temp = num_A % divisor (result in R0)
// ADDR 8:  CMP   R0, R0          ; check if R0 == 0
// (zero_flag trick | see note*)
//          ; *actual zero check: CMP R0 with constant 0 via IMM path
// ADDR 9:  BEQ   11              ; if remainder==0 → branch to ADDR 11
// (inc factcnt)
// ADDR 10: INC   R3              ; divisor++ (no factor found)
// ADDR 11: INC   R5              ; factcnt_A++ (factor found for A)
// ADDR 12: INC   R3              ; divisor++
//
// ; Check B in same loop
// ADDR 13: CMP   R3, R7          ; compare divisor vs half_B
// ADDR 14: BGT   17              ; if divisor > half_B → exit loop
// ADDR 15: MOD   R0, R2, R3      ; temp = num_B % divisor
// ADDR 16: BEQ   18              ; if remainder==0 → inc factcnt_B
// ADDR 17: INC   R6              ; factcnt_B++
//
// ; Loop back
// ADDR 18: BGT   5               ; jump back to ADDR 5 (loop top)
//
// ; STAGE MEM: Accumulate
// ADDR 19: MOV   R0, R5          ; move factcnt_A to output path
// ADDR 20: MOV   R7, R6          ; move factcnt_B to output path
//
// ; STAGE WB: Write Back
// ADDR 21: CMP   R5, R0          ; check factcnt_A == 0?
// ADDR 22: BEQ   24              ; if 0 → prime_A = 1 (branch to set)
// ADDR 23: OUT   R5              ; output not-prime (factcnt > 0)
// ADDR 24: OUT   R6              ; output prime_A result
// ADDR 25: HALT                  ; done | assert valid_out
// =====

'timescale 1ns / 1ps

module instr_mem (
    input wire [4:0]  addr,        // PC address (supports 32 instructions)
    output reg [15:0] instr        // 16-bit instruction word
);

// Instruction encoding helper:
// {opcode[3:0], rd[2:0], rs1[2:0], rs2[2:0], imm[2:0]}

```

```
always @(*) begin
    case (addr)
        // INIT (maps to IF + ID stages)
        // LOAD R3, 2 → opcode=0001, rd=011, rs1=000, rs2=000, imm=010
        5'd0: instr = 16'b0001_011_000_000_010;

        // SHR R4, R1 → opcode=0110, rd=100, rs1=001, rs2=000, imm=000
        5'd1: instr = 16'b0110_100_001_000_000;

        // SHR R7, R2 → opcode=0110, rd=111, rs1=010, rs2=000, imm=000
        5'd2: instr = 16'b0110_111_010_000_000;

        // LOAD R5, 0 → opcode=0001, rd=101, rs1=000, rs2=000, imm=000
        5'd3: instr = 16'b0001_101_000_000_000;

        // LOAD R6, 0 → opcode=0001, rd=110, rs1=000, rs2=000, imm=000
        5'd4: instr = 16'b0001_110_000_000_000;

        // EX LOOP (trial division)
        // CMP R3, R4 → opcode=0111, rd=000, rs1=011, rs2=100, imm=000
        5'd5: instr = 16'b0111_000_011_100_000;

        // BGT addr=16 → opcode=1001, rd=000, rs1=000, rs2=000, imm=|
        // (branch target encoded in rs1+rs2+imm = 5 bits)
        5'd6: instr = 16'b1001_00000_10000_0; // branch to addr 16

        // MOD R0, R1, R3→opcode=0101, rd=000, rs1=001, rs2=011, imm=000
        5'd7: instr = 16'b0101_000_001_011_000;

        // CMP R0, #0 → opcode=0111, rd=000, rs1=000, rs2=000, imm=000
        5'd8: instr = 16'b0111_000_000_000_000;

        // BEQ addr=11 → opcode=1000, target=01011
        5'd9: instr = 16'b1000_00000_01011_0;

        // INC R3 → opcode=1010, rd=011
        5'd10: instr = 16'b1010_011_000_000_000;

        // INC R5 → factcnt_A++
        5'd11: instr = 16'b1010_101_000_000_000;

        // INC R3 → divisor++
        5'd12: instr = 16'b1010_011_000_000_000;

        // CMP R3, R7 → divisor vs half_B
        5'd13: instr = 16'b0111_000_011_111_000;

        // BGT addr=17
```

```
5'd14: instr = 16'b1001_00000_10001_0;

// MOD  R0, R2, R3 → num_B % divisor
5'd15: instr = 16'b0101_000_010_011_000;

// BEQ  addr=18
5'd16: instr = 16'b1000_00000_10010_0;

// INC  R6          → factcnt_B++
5'd17: instr = 16'b1010_110_000_000_000;

// BGT  addr=5      → loop back
5'd18: instr = 16'b1001_00000_00101_0;

// MEM stage
// MOV  R0, R5
5'd19: instr = 16'b0010_000_101_000_000;

// MOV  R7, R6
5'd20: instr = 16'b0010_111_110_000_000;

// WB stage
// CMP  R5, #0
5'd21: instr = 16'b0111_000_101_000_000;

// BEQ  addr=24
5'd22: instr = 16'b1000_00000_11000_0;

// OUT  R5
5'd23: instr = 16'b1011_000_101_000_000;

// OUT  R6
5'd24: instr = 16'b1011_000_110_000_000;

// HALT
5'd25: instr = 16'b1111_000_000_000_000;

    default: instr = 16'b0000_000_000_000_000; // NOP
endcase
end

endmodule
```

3. Register File

```
// =====
// reg_file.v | 8 x 8-bit Register File
```

```
// R0 = zero register (reads always 0, but used as temp in ALU output)
// R1 = num_A (loaded from input port at start)
// R2 = num_B
// R3 = divisor
// R4 = half_A
// R5 = factcnt_A
// R6 = factcnt_B
// R7 = half_B / scratch
// =====

`timescale 1ns / 1ps

module reg_file (
    input wire      clk,
    input wire      rst,
    // Write port
    input wire      we,          // write enable
    input wire [2:0] wr_addr,    // destination register
    input wire [7:0] wr_data,    // data to write
    // Read ports (two async read ports)
    input wire [2:0] rd_addr1,
    input wire [2:0] rd_addr2,
    output wire [7:0] rd_data1,
    output wire [7:0] rd_data2
);

    reg [7:0] regs [0:7];
    integer i;

    // Asynchronous read
    assign rd_data1 = (rd_addr1 == 3'd0) ? 8'd0 : regs[rd_addr1];
    assign rd_data2 = (rd_addr2 == 3'd0) ? 8'd0 : regs[rd_addr2];

    // Synchronous write
    always @(posedge clk) begin
        if (rst) begin
            for (i = 0; i < 8; i = i + 1)
                regs[i] <= 8'd0;
        end else if (we && wr_addr != 3'd0) begin
            regs[wr_addr] <= wr_data;
        end
    end

endmodule
```

4. Pipeline Processor File

```
'timescale 1ns / 1ps

// Pull in opcode definitions
#include "isa_pkg.v"

module prime_pipeline (
    input wire      clk,
    input wire      rst,
    input wire [7:0] num_A,
    input wire [7:0] num_B,
    input wire      start,

    output reg      is_prime_A,
    output reg      is_prime_B,
    output reg      valid_out,
    output reg [2:0] stage_dbg
    // shows current PC stage band for SimVision
);

// =====
// PROGRAM COUNTER
// =====
reg [4:0] PC;
wire [15:0] instr_if;      // raw instruction from memory

// =====
// INSTANTIATE INSTRUCTION MEMORY
// =====
instr_mem IMEM (
    .addr (PC),
    .instr (instr_if)
);

// =====
// INSTANTIATE REGISTER FILE
// =====
// WB stage drives these
reg      rf_we;
reg [2:0] rf_wr_addr;
reg [7:0] rf_wr_data;
// ID stage reads these
wire [7:0] rf_rs1_data, rf_rs2_data;
reg [2:0] id_rs1_addr, id_rs2_addr;

reg_file REGFILE (
    .clk      (clk),
    .rst      (rst),
    .we       (rf_we),
```

```
.wr_addr  (rf_wr_addr),
.wr_data  (rf_wr_data),
.rd_addr1 (id_rs1_addr),
.rd_addr2 (id_rs2_addr),
.rd_data1 (rf_rs1_data),
.rd_data2 (rf_rs2_data)
);

// =====
// IF / ID PIPELINE REGISTER
// =====
reg [15:0] id_instr;
reg [4:0]  id_pc;
reg       id_valid;

// =====
// ID / EX PIPELINE REGISTER
// =====
reg [3:0]  ex_opcode;
reg [2:0]  ex_rd;
reg [7:0]  ex_rs1_data, ex_rs2_data;
reg [7:0]  ex_imm;
reg [4:0]  ex_br_target;
reg       ex_valid;
reg [4:0]  ex_pc;

// =====
// EX / MEM PIPELINE REGISTER
// =====
reg [3:0]  mem_opcode;
reg [2:0]  mem_rd;
reg [7:0]  mem_alu_result;
reg       mem_we;           // write-enable for register file
reg       mem_valid;
reg       mem_zero_flag;    // from ALU CMP
reg       mem_branch_taken;
reg [4:0]  mem_br_target;

// =====
// MEM / WB PIPELINE REGISTER
// =====
reg [3:0]  wb_opcode;
reg [2:0]  wb_rd;
reg [7:0]  wb_result;
reg       wb_we;
reg       wb_valid;

// =====
```

```
// ALU
// =====
reg [7:0] alu_a, alu_b;
reg [3:0] alu_op;
reg [7:0] alu_result;
reg      alu_zero;          // 1 if result == 0 or a == b

always @(*) begin
    alu_result = 8'd0;
    alu_zero   = 1'b0;
    case (alu_op)
        'OP_ADD: begin alu_result = alu_a + alu_b; end
        'OP_SUB: begin alu_result = alu_a - alu_b; alu_zero =(alu_result==0);
        end
        'OP_MOD: begin alu_result = (alu_b != 0) ? (alu_a % alu_b) : 8'd0;
                    alu_zero   = (alu_result == 0); end
        'OP_SHR: begin alu_result = alu_a >> 1; end
        'OP_CMP: begin alu_result = 8'd0; alu_zero = (alu_a == alu_b); end
        'OP_INC: begin alu_result = alu_a + 8'd1; end
        'OP_MOV: begin alu_result = alu_a; end
        'OP_LOAD: begin alu_result = alu_b; end    // b carries immediate
        default: begin alu_result = 8'd0; end
    endcase
end

reg halted;
reg zero_flag;

// =====
// PIPELINE | all stages in one clocked block
// =====
always @(posedge clk) begin

    if (rst) begin
        PC          <= 5'd0;
        halted      <= 1'b0;
        valid_out    <= 1'b0;
        is_prime_A   <= 1'b0;
        is_prime_B   <= 1'b0;
        zero_flag    <= 1'b0;
        stage_dbg    <= 3'd0;

        // Pipeline register reset
        id_instr     <= 16'd0; id_valid  <= 1'b0;
        ex_opcode    <= 4'd0;  ex_valid  <= 1'b0;
        mem_opcode   <= 4'd0;  mem_valid <= 1'b0;
        wb_opcode    <= 4'd0;  wb_valid  <= 1'b0;
    end
end
```

```
// Load num_A and num_B into R1 and R2 via WB path at reset
rf_we      <= 1'b0;
rf_wr_addr  <= 3'd0;
rf_wr_data  <= 8'd0;

end else if (!halted) begin

    if (start && PC == 5'd0) begin
        // Cycle 1 of start: write R1 = num_A
        rf_we      <= 1'b1;
        rf_wr_addr <= 3'd1;
        rf_wr_data <= num_A;
    end

    // =====
    // STAGE 1 | IF : Instruction Fetch
    // =====

    // Latch into IF/ID register
    id_instr <= instr_if;
    id_pc    <= PC;
    id_valid <= 1'b1;
    stage_dbg <= 3'd1;

    // =====
    // STAGE 2 | ID : Instruction Decode
    // =====
    begin
        // Decode fields from id_instr
        ex_opcode <= id_instr[15:12];
        ex_rd     <= id_instr[11:9];
        id_rs1_addr <= id_instr[8:6];
        id_rs2_addr <= id_instr[5:3];
        ex_imm     <= {5'd0, id_instr[2:0]}; // zero-extend 3-bit imm
        ex_br_target <= id_instr[7:3]; // branch uses [7:3] as addr
        ex_rs1_data <= rf_rs1_data;
        ex_rs2_data <= rf_rs2_data;
        ex_pc       <= id_pc;
        ex_valid    <= id_valid;
    end

    // =====
    // STAGE 3 | EX : Execute (ALU)
    // =====
    begin
        alu_op = ex_opcode;
```

```

// Select ALU inputs based on opcode
case (ex_opcode)
    'OP_LOAD: begin alu_a = 8'd0;          alu_b = ex_imm; end
    'OP_SHR:  begin alu_a = ex_rs1_data; alu_b = 8'd0; end
    'OP_INC:  begin alu_a = ex_rs1_data; alu_b = 8'd0; end
    'OP_MOV:  begin alu_a = ex_rs1_data; alu_b = 8'd0; end
    'OP_CMP:  begin alu_a = ex_rs1_data; alu_b = ex_rs2_data; end
    'OP_MOD:  begin alu_a = ex_rs1_data; alu_b = ex_rs2_data; end
    'OP_ADD:  begin alu_a = ex_rs1_data; alu_b = ex_rs2_data; end
    'OP_SUB:  begin alu_a = ex_rs1_data; alu_b = ex_rs2_data; end
    default:  begin alu_a = ex_rs1_data; alu_b = ex_rs2_data; end
endcase

// Branch detection
mem_branch_taken <= 1'b0;
mem_br_target    <= ex_br_target;

if (ex_opcode == 'OP_BEQ && zero_flag) begin
    mem_branch_taken <= 1'b1;
end
if (ex_opcode == 'OP_BGT && (ex_rs1_data > ex_rs2_data)) begin
    mem_branch_taken <= 1'b1;
end

mem_opcode    <= ex_opcode;
mem_rd        <= ex_rd;
mem_alu_result <= alu_result;
mem_zero_flag <= alu_zero;
mem_we        <= (ex_opcode != 'OP_CMP  &&
                  ex_opcode != 'OP_BEQ  &&
                  ex_opcode != 'OP_BGT  &&
                  ex_opcode != 'OP_OUT  &&
                  ex_opcode != 'OP_HALT &&
                  ex_opcode != 'OP_NOP);
mem_valid     <= ex_valid;
end

// =====
// STAGE 4 | MEM : Memory / Accumulate
// =====
begin
    // Update zero_flag from CMP result
    if (mem_opcode == 'OP_CMP)
        zero_flag <= mem_zero_flag;

    // Handle OUT instruction | drive output pins
    if (mem_opcode == 'OP_OUT) begin
        // OUT R5 → is_prime_A, OUT R6 → is_prime_B

```

```
        if (mem_rd == 3'd0 && mem_alu_result == 3'd5)
            is_prime_A <= (mem_alu_result == 8'd0) ? 1'b1 : 1'b0;
        if (mem_rd == 3'd0 && mem_alu_result == 3'd6)
            is_prime_B <= (mem_alu_result == 8'd0) ? 1'b1 : 1'b0;
    end

    // HALT | freeze and signal done
    if (mem_opcode == 'OP_HALT) begin
        halted    <= 1'b1;
        valid_out <= 1'b1;
    end

    wb_opcode <= mem_opcode;
    wb_rd     <= mem_rd;
    wb_result <= mem_alu_result;
    wb_we     <= mem_we;
    wb_valid  <= mem_valid;
end

// =====
// STAGE 5 | WB : Write Back
// =====
begin
    if (wb_we && wb_valid) begin
        rf_we      <= 1'b1;
        rf_wr_addr <= wb_rd;
        rf_wr_data <= wb_result;
    end else begin
        rf_we <= 1'b0;
    end

    // Final primality output (WB reads R5/R6 at HALT)
    if (wb_opcode == 'OP_HALT) begin
        // R5 = factcnt_A, R6 = factcnt_B at this point
        // The OUT instructions before HALT already set is_prime
        stage_dbg <= 3'd5;
    end
end

// =====
// PC UPDATE | after all stages, advance or branch
// =====
if (mem_branch_taken)
    PC <= mem_br_target;
else
    PC <= PC + 5'd1;

end // !halted
```

```
end
```

```
endmodule
```

5. Testbench File

```
'timescale 1ns / 1ps
```

```
module tb_prime_pipeline;
```

```
    reg        clk, rst, start;
    reg [7:0] num_A, num_B;
    wire        is_prime_A, is_prime_B, valid_out;
    wire [2:0] stage_dbg;
```

```
    // DUT
```

```
    prime_pipeline DUT (
        .clk      (clk),
        .rst      (rst),
        .num_A    (num_A),
        .num_B    (num_B),
        .start    (start),
        .is_prime_A(is_prime_A),
        .is_prime_B(is_prime_B),
        .valid_out (valid_out),
        .stage_dbg (stage_dbg)
    );
```

```
    // Clock: 10 ns period
```

```
    initial clk = 0;
    always #5 clk = ~clk;
```

```
    // Stimulus
```

```
    initial begin
```

```
        // Waveform dump for SimVision
        $dumpfile("prime_pipeline.vcd");
        $dumpvars(0, tb_prime_pipeline);
```

```
        // Test 1: num_A=7 (prime), num_B=9 (not prime)
```

```
        rst = 1; start = 0; num_A = 8'd7; num_B = 8'd9;
        #20; rst = 0;
        #10; start = 1;
        #10; start = 0;
```

```
        // Wait for valid_out
```

```
        wait(valid_out == 1);
        #10;
```

```
        $display("Test1: num_A=%0d is_prime_A=%b | num_B=%0d is_prime_B=%b",
```

```
        num_A, is_prime_A, num_B, is_prime_B);
// Expected: is_prime_A=1, is_prime_B=0

// Test 2: num_A=13 (prime), num_B=15 (not prime)
rst = 1; #20; rst = 0;
num_A = 8'd13; num_B = 8'd15;
#10; start = 1; #10; start = 0;
wait(valid_out == 1); #10;
$display("Test2: num_A=%0d is_prime_A=%b | num_B=%0d is_prime_B=%b",
        num_A, is_prime_A, num_B, is_prime_B);
// Expected: is_prime_A=1, is_prime_B=0

// Test 3: num_A=2 (prime edge case), num_B=1 (not prime)
rst = 1; #20; rst = 0;
num_A = 8'd2; num_B = 8'd1;
#10; start = 1; #10; start = 0;
wait(valid_out == 1); #10;
$display("Test3: num_A=%0d is_prime_A=%b | num_B=%0d is_prime_B=%b",
        num_A, is_prime_A, num_B, is_prime_B);
// Expected: is_prime_A=1, is_prime_B=0

// Test 4: num_A=17 (prime), num_B=25 (not prime)
rst = 1; #20; rst = 0;
num_A = 8'd17; num_B = 8'd25;
#10; start = 1; #10; start = 0;
wait(valid_out == 1); #10;
$display("Test4: num_A=%0d is_prime_A=%b | num_B=%0d is_prime_B=%b",
        num_A, is_prime_A, num_B, is_prime_B);

#50;
$display("All tests complete.");
$finish;
end

// Timeout watchdog
initial begin
    #100000;
    $display("TIMEOUT | simulation ended forcefully");
    $finish;
end

endmodule
```

6. FPGA Pin Mapping File

```
## constraints.xdc | Xilinx XDC for Basys3 / Nexys4
## Adjust pin names to match your exact board
```

```
## Clock (100 MHz onboard)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 [get_ports clk]

## Reset | Button (active high)
set_property PACKAGE_PIN U18 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports rst]

## Start | Button
set_property PACKAGE_PIN T18 [get_ports start]
set_property IOSTANDARD LVCMOS33 [get_ports start]

## num_A[7:0] | Switches SW7..SW0
set_property PACKAGE_PIN V17 [get_ports {num_A[0]}]
set_property PACKAGE_PIN V16 [get_ports {num_A[1]}]
set_property PACKAGE_PIN W16 [get_ports {num_A[2]}]
set_property PACKAGE_PIN W17 [get_ports {num_A[3]}]
set_property PACKAGE_PIN W15 [get_ports {num_A[4]}]
set_property PACKAGE_PIN V15 [get_ports {num_A[5]}]
set_property PACKAGE_PIN W14 [get_ports {num_A[6]}]
set_property PACKAGE_PIN W13 [get_ports {num_A[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {num_A[*]}]

## num_B[7:0] | Switches SW15..SW8
set_property PACKAGE_PIN V2 [get_ports {num_B[0]}]
set_property PACKAGE_PIN T3 [get_ports {num_B[1]}]
set_property PACKAGE_PIN T2 [get_ports {num_B[2]}]
set_property PACKAGE_PIN R3 [get_ports {num_B[3]}]
set_property PACKAGE_PIN W2 [get_ports {num_B[4]}]
set_property PACKAGE_PIN U1 [get_ports {num_B[5]}]
set_property PACKAGE_PIN T1 [get_ports {num_B[6]}]
set_property PACKAGE_PIN R2 [get_ports {num_B[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {num_B[*]}]

## Outputs | LEDs
set_property PACKAGE_PIN U16 [get_ports is_prime_A]
set_property IOSTANDARD LVCMOS33 [get_ports is_prime_A]

set_property PACKAGE_PIN E19 [get_ports is_prime_B]
set_property IOSTANDARD LVCMOS33 [get_ports is_prime_B]

set_property PACKAGE_PIN U19 [get_ports valid_out]
set_property IOSTANDARD LVCMOS33 [get_ports valid_out]
```

Procedure

The procedure followed for the implementation and simulation of the Prime Number Detection System is as follows:

1. Create a new project folder for the Verilog files.
2. Write the Verilog code for the Program Counter, Instruction Memory, IF/ID pipeline register, Register File, ID/EX pipeline register, ALU, and EX/MEM pipeline register.
3. Create the top module to connect all the pipeline stages.
4. Write a testbench to generate clock and reset signals.
5. Compile the Verilog files using Cadence NCLaunch or Xcelium.
6. Simulate the design and open the waveform window in SimVision.
7. Add important signals such as clock, reset, program counter, instruction, register data, ALU result, and prime flag.
8. Verify the waveform output for each pipeline stage.
9. Check whether the ALU performs the modulus operation correctly.
10. Observe the prime flag to determine whether the given number is prime or not.

Expected Output

The expected output of the simulation is the correct working of each pipeline stage. The clock signal should toggle continuously and the reset signal should initialize the processor at the beginning of simulation.

The Program Counter is expected to increment by 4 in every clock cycle. The instruction memory should fetch instructions according to the current PC value, and the IF/ID pipeline register should store the fetched instruction with one clock cycle delay.

In the decode stage, the register file should read the required register values. These values should be passed to the ID/EX pipeline register. In the execution stage, the ALU should perform the modulus operation on the input number and divisor.

If the remainder is zero, the prime flag should indicate that the number is not prime. If the remainder is non-zero for all divisors, the prime flag should indicate that the number is prime.

- Clock signal toggles continuously.
- Reset initializes all pipeline registers.
- Program Counter increments as 0, 4, 8, 12, ...
- Instructions are fetched from instruction memory.
- Pipeline registers show one-cycle delay.
- ALU performs modulus operation.
- Prime flag indicates prime or not-prime result.

Waveform Observation Points

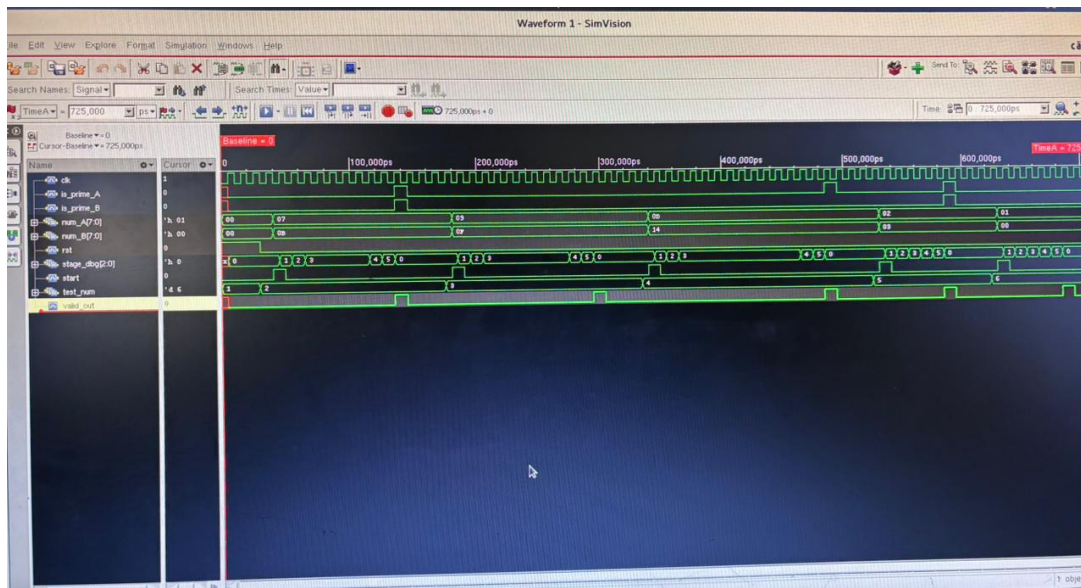


Figure 2: Cadence Simulation Waveforms of 5-Stage Pipelined Prime Number Detection Processor

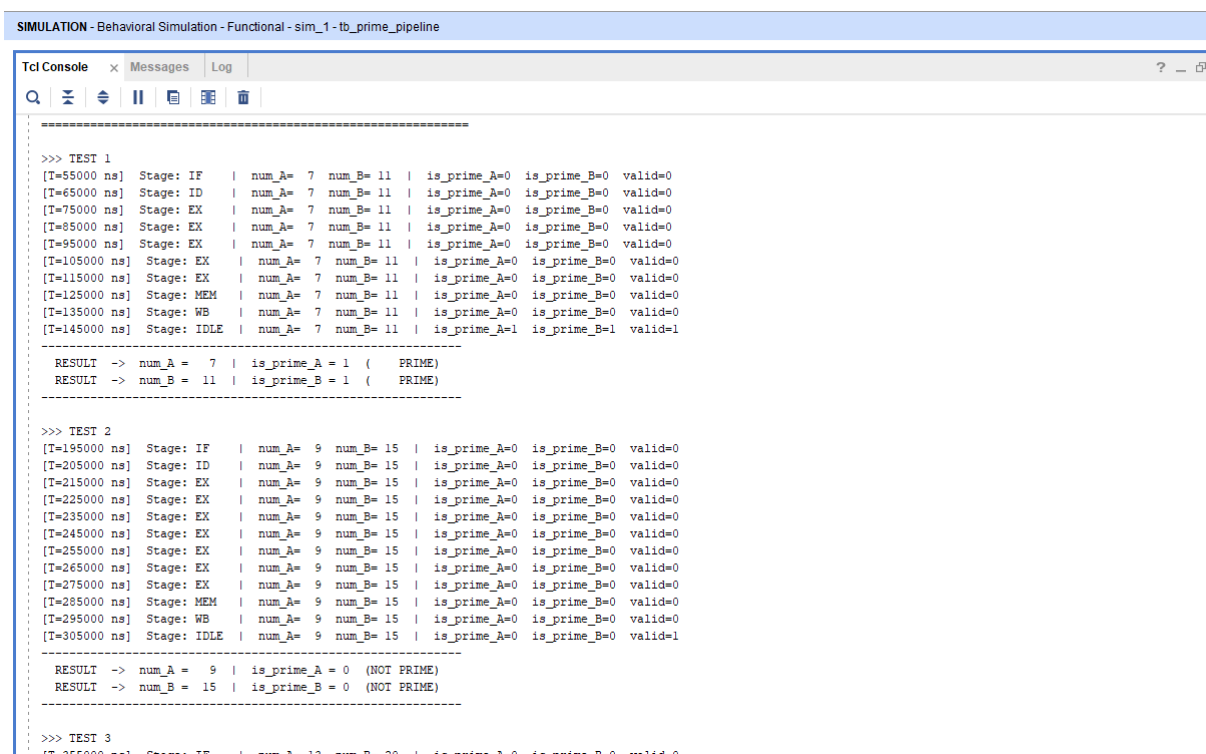


Figure 3: Console Window Output

```

RESULT -> num_D = 13 | is_prime_D = 0 (NOT PRIME)
-----
>>> TEST 3
[T=355000 ns] Stage: IF | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=365000 ns] Stage: ID | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=375000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=385000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=395000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=405000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=415000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=425000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=435000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=445000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=455000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=465000 ns] Stage: EX | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=475000 ns] Stage: MEM | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=485000 ns] Stage: WB | num_A= 13 num_B= 20 | is_prime_A=0 is_prime_B=0 valid=0
[T=495000 ns] Stage: IDLE | num_A= 13 num_B= 20 | is_prime_A=1 is_prime_B=0 valid=1

RESULT -> num_A = 13 | is_prime_A = 1 ( PRIME)
RESULT -> num_B = 20 | is_prime_B = 0 (NOT PRIME)

```

Figure 4: Console Window Output

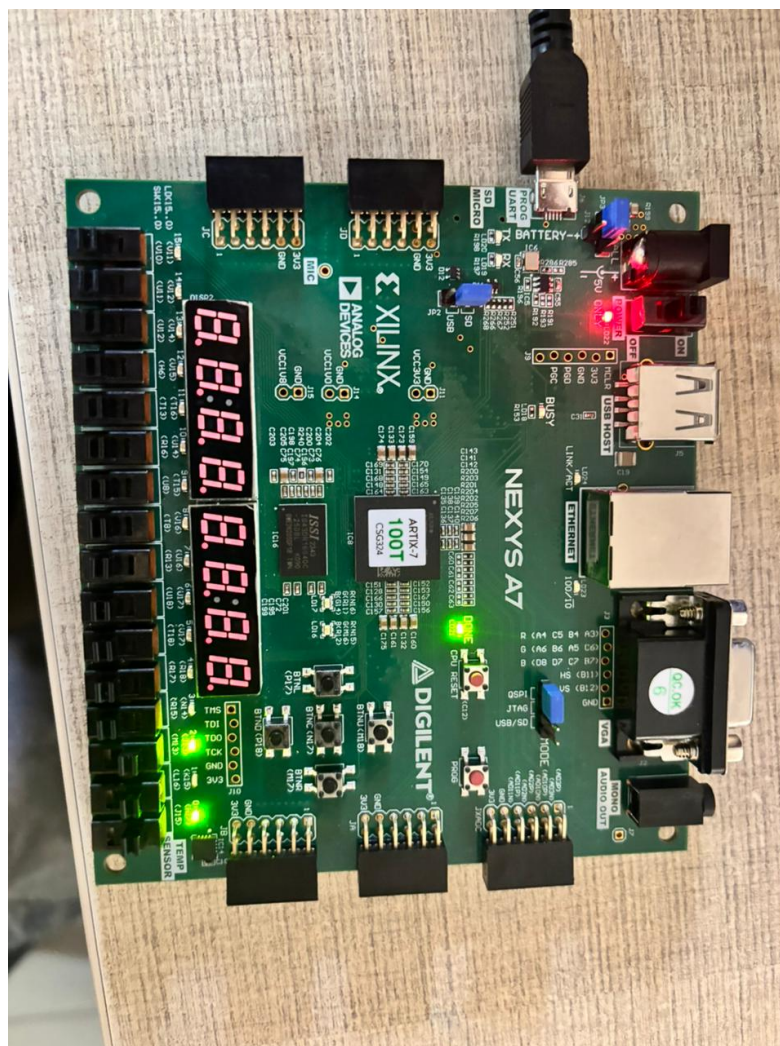


Figure 5: Hardware Output on FPGA Board

Result and Conclusion

The Prime Number Detection System using a 5-Stage Pipelined Processor was successfully designed and simulated using Verilog HDL in Cadence. The simulation verified the correct operation of the pipeline stages including Instruction Fetch, Instruction Decode, and Execution.

The Program Counter incremented correctly, instructions passed through the pipeline registers properly, and the ALU successfully performed the modulus operation for prime number detection. The prime flag was generated based on the remainder value.

The generated waveforms confirmed the correct functioning of the program counter, instruction memory, pipeline registers, register file, and ALU. This project helped in understanding RTL design, pipelined processor implementation, and simulation-based verification of digital systems.

Reference

1. California State University ScholarWorks. Available at:
<https://scholarworks.calstate.edu/downloads/pz50h4333>
2. IEEE Xplore Digital Library. Available at:
<https://ieeexplore.ieee.org/abstract/document/11081376/>
3. IEEE Xplore Digital Library. Available at:
<https://ieeexplore.ieee.org/abstract/document/11182670/>